

PROJECT

7.4

CONTINUOUS INTEGRATION WITH CODEBUILD

DEVOPS SERIES

Prepared by:
Amrize JK
Certified Solutions Architect Associate

Objective

This project sets up an automated workflow to build and test a web application using AWS CodeBuild, triggered by changes pushed to a connected GitHub repository.

IAM manages secure access, and CodeArtifact provides build dependencies. The setup creates a simple and reliable CI process for faster and safer code updates.

Services

Amazon EC2 - Hosts the dev environment to prepare and test your app setup.

AWS IAM - Controls access to CodeBuild, CodeArtifact, and GitHub connections securely.

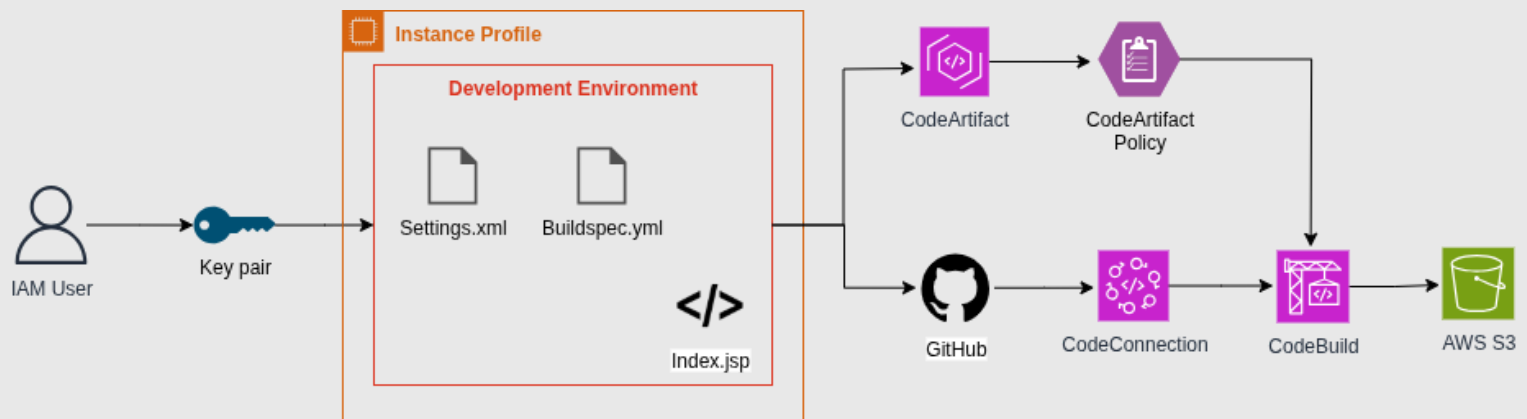
AWS CodeBuild - Automatically builds and tests your application after every code update.

AWS CodeArtifact - Provides private storage and access to dependencies during builds.

GitHub - Stores your application source code and triggers CI workflows.

AWS CLI - Enables command-line setup and integration of all build and deployment steps.

Architecture Diagram



This architecture shows an EC2 environment where code and config files (index.jsp, settings.xml, buildspec.yml) are managed. It connects to GitHub and CodeArtifact, with IAM handling access. CodeBuild builds the app via CodeConnection and stores the output in S3.

Steps to Implement

1. Launch and set up your development instance:

Start an EC2 instance, connect using VSCode Remote-SSH, and prepare your app environment for development.

2. Initialize your Git project and connect GitHub:

Set up Git on EC2, create a GitHub repo, and link it using git remote add origin for version control.

3. Generate and use a GitHub PAT:

Create a Personal Access Token on GitHub, and use it during git push to authenticate securely from EC2.

4. Create and configure settings.xml for CodeArtifact:

Generate a settings.xml file with authentication details for accessing your CodeArtifact repository.

5. Set up a CodeArtifact repository and domain:

Create a private repository in AWS CodeArtifact to manage and retrieve app dependencies securely.

6. Create an IAM policy and attach it to EC2 role:

Define and attach a policy that grants access to CodeArtifact and CodeBuild services from your EC2 instance.

7. Build, test, and push your code using CodeBuild:

Create a CodeBuild project, connect it to GitHub, run builds, and verify outputs and artifacts for success.

Story Time

EC2 (The Cloud Castle for Your App):

EC2 is your grand castle in the cloud where your app is crafted. You launch it, equip it with tools, and access it from afar like a wizard using VSCode's Remote-SSH portal.

GitHub (The Code Scroll Library):

GitHub is your enchanted scroll library where all your code spells are stored. Every push to it updates the magic for everyone in the kingdom to use and improve.

settings.xml (The Hidden Access Spell):

settings.xml is your secret incantation scroll that holds the access codes for the Potion Vault (CodeArtifact). Without it, no potions can be fetched!

GitHub PAT (The Secret Key of Trust):

The Personal Access Token is your golden key that proves your identity to the scroll library. Without it, your scrolls can't be updated from the castle.

CodeArtifact (The Potion Vault):

CodeArtifact is your vault of powerful potions (packages). You pull out these ingredients during builds and use the settings.xml scroll to gain secure access.

CodeBuild (The Auto-Crafting Wizard):

CodeBuild is your tireless auto-crafting wizard. Every time you update your scrolls, it wakes up, grabs the latest code, mixes the potions, and builds your magical artifact—no wand needed!

BY
AMRIZE J.K

THANK YOU!

